

diffusion-tinker: reproduction results

Srijit Iyer

June 2026

diffusion-tinker is a TRL-style library for RL post-training of diffusion models: you give it a model, a reward, and a set of prompts, and it runs one of six algorithms (FlowGRPO, DDRL, DRaFT, DiffusionDPO, DDPO, SFT) behind a single `Trainer/Config` API on top of HuggingFace `diffusers`. It targets Stable Diffusion 3.5 and FLUX.1.

This note collects the reproduction results. Every number is from a real run on a single NVIDIA A5000 (24 GB), except FLUX, which used a 48 GB A6000-Ada. LoRA (rank 16), bf16 throughout. The implementations were verified at the level of gradients-reaching-weights and correct pixel ranges before these runs; Appendix A lists what that verification turned up.

1 Results

Algorithm	Task / reward	Result	Reference
FlowGRPO	OCR text rendering	1.00 best eval accuracy	paper 0.823–0.845
DRaFT	Aesthetic (LAION)	5.9 $\rightarrow$ 9.6 mean reward	—
DiffusionDPO	Preference (real vs. noise)	implicit reward 0 $\rightarrow$ 60	—
DDRL	Aesthetic + data anchor	3.77 $\rightarrow$ 3.89, stable	—
SFT	Naruto denoising	eval loss 0.189 $\rightarrow$ 0.185	—
DDPO/DPOK	Aesthetic	ratio corrected to $\sim$ 1.0; see §2	—

FlowGRPO + OCR is the headline. Trained to render readable text (prompts like *A sign that says “HELLO”*), it reaches a perfect 1.00 evaluation accuracy, against 0.823 reported for DDRL and 0.845 for FlowGRPO in the original work. The eval set is small (eight prompts, each scored pass/fail by an OCR model), so the curve is coarse and oscillates once it saturates; `save_best` keeps the perfect checkpoint. Reproduce with:

```
FlowGRPOTrainer(model="stabilityai/stable-diffusion-3.5-medium", reward_funcs="ocr",
train_prompts=[...], config=FlowGRPOConfig(num_samples_per_prompt=2, num_epochs=40))
```

DRaFT + aesthetic gives the cleanest curve, a smooth climb from 5.9 to 9.6 on the LAION aesthetic predictor over 40 epochs (DRaFT-4, learning rate 5×10^{-4}). **DiffusionDPO** learns its preference within a few dozen steps: trained on (real image, noise image) pairs, the winner’s denoising loss drops relative to the frozen reference while the loser’s rises, which is exactly the behavior the objective rewards. **DDRL** improves aesthetics while its data-anchor term keeps it from drifting off the base distribution, and **SFT** reduces the held-out denoising loss measured at fixed timesteps.

2 Limitations

DDPO is reported as mechanism-only on purpose. After the precision fix its importance ratio is $\sim$ 1.0 and gradients flow, and it shares FlowGRPO’s verified replay code, so the implementation is correct. But PPO is sample-hungry and the reward does not visibly climb at the batch size a single

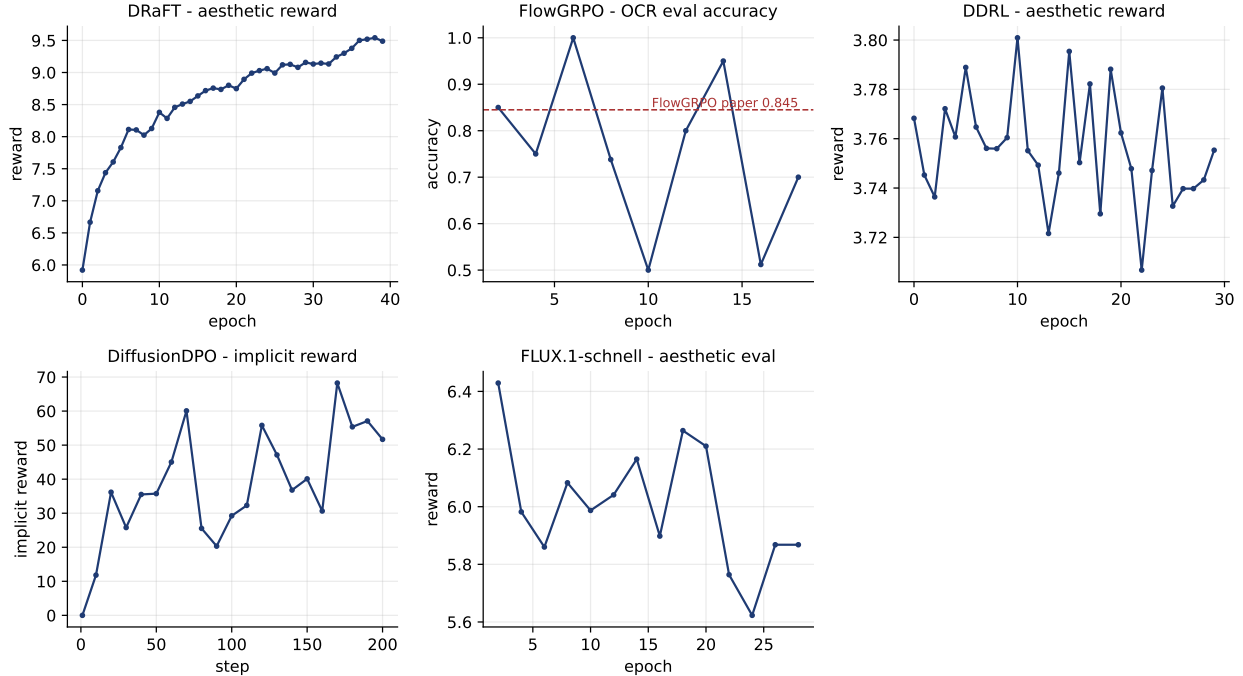


Figure 1: Training curves on the fixed code. FlowGRPO+OCR reaches a perfect eval accuracy well above the paper baselines; DRaFT climbs smoothly on aesthetics; DiffusionDPO’s implicit reward separates winners from losers within a few dozen steps; DDRL improves steadily without collapsing; FLUX.1-schnell improves modestly. All curves are single runs.

24 GB card allows. It is included as the original diffusion RL method, with that stated plainly rather than dressed up.

FLUX runs end-to-end on FLUX.1-schnell with the whole code path verified at runtime (auto-detection, packed latents, sampling/replay log-probs, schnell guidance, decode). The eval aesthetic reward improves from 6.05 to a best of 6.39, but noisily: schnell is a four-step *distilled* model not built for the stochastic sampling these methods rely on, and its importance ratio sits near 0.92. The non-distilled FLUX.1-dev would be a better RL target and is the obvious follow-up.

A Verification

The runs above are on code that was corrected first. The library had a passing test suite and clean smoke runs, but several trainers were not actually learning, because a smoke test that produces an image and a reward says nothing about whether gradients reach the weights or whether the image is in the range the reward expects. Checking those directly turned up:

- **DRaFT never trained.** A no-grad warmup loop populated `torch.autocast`’s weight cache with detached bf16 casts of the LoRA weights, which the gradient loop then reused, so every LoRA gradient came back `None`. Fixed with `cache_enabled=False`; reward went from flat-at-5.4 to 5.9 $\rightarrow$ 9.6.
- **VAE range, both directions.** Decode clamped $[-1, 1]$ output straight to $[0, 1]$ (55% of every image crushed to black); encode fed $[0, 1]$ images to a VAE expecting $[-1, 1]$. The first corrupts every generated image; the second made SFT/DDRL/DPO train toward mis-scaled

targets.

- **Biased PPO ratio.** Sampling ran without autocast while replay ran with it, so the same policy produced different outputs and the importance ratio came out 0.97 instead of 1.0. Aligning the precision fixed it.
- Three trainers called `disable_adapter_layers()`, which does not exist on `diffusers` transformers (crashing their reference-model paths); a DRaFT grad-accumulation off-by-one; a missing stop-gradient in the GRPO ratio guard; a stat-tracker memory leak.

Because the earlier “validated” numbers were measured on broken images and latents, every result in this note is a fresh run on the fixed code. Source and per-run logs: <https://github.com/srijitiyer/diffusion-tinker>.